

Meeting Notes & Design Document

Topic: Using Persistent Maps for Next-Use Analysis (NUA)

1. Context

Our current SSA-based Next Use Analysis (NUA) maintains **full per-instruction snapshots** of `VRegDistances` to allow constant-time queries. This guarantees correctness but consumes $\Theta(I \cdot \bar{K})$ memory (I = instructions, $\bar{K}$ = average number of live virtual registers). For large functions this is prohibitive.

2. Proposed Optimization: Persistent Map (Functional Data Structure)

What “Persistent” Means

- **Not disk persistence.** Here, “persistent” means *immutable but versioned*.
- Every update returns a new **version** (root pointer), but shares most structure with previous versions.
- Old versions remain valid; they are not destroyed.

Why This Helps

- Instead of copying the whole map at each instruction, we reuse 95–99% of the structure.
- **Memory** now grows with the number of *changes* (operand touches), not with $I \cdot \bar{K}$.
- **Queries** are still $O(1)$: each instruction just stores a pointer to the map version valid at that point.

3. LLVM Infrastructure: `ImmutableMap`

LLVM already provides `ImmutableMap` in `llvm/ADT/ImmutableMap.h`. It implements a persistent balanced tree with structural sharing.

We can model: - Level 1: `VReg -> Map<LaneMask, StoredVal>` - Level 2: `LaneMask -> int64_t`
(signed stored distance)

Thus each instruction stores a pointer to a **persistent map root**. Updates (insert/clear) return a new root; old ones remain valid.

4. Prototype API Sketch

```
// Lane mask key
struct LaneMaskKey {
    LaneBitmask Mask;
    void Profile(FoldingSetNodeID &ID) const {
        ID.AddInteger((uint64_t)Mask.getAsInteger());
    }
};
```

```

// VReg key
struct VRegKey {
    unsigned VReg;
    void Profile(FoldingSetNodeID &ID) const { ID.AddInteger(VReg); }
};

// Maps
using PM_Lane = ImmutableMap<LaneMaskKey, int64_t>;
using PM_VReg = ImmutableMap<VRegKey, PM_Lane>;

struct PMapFactories {
    ImmutableMapFactory<LaneMaskKey, int64_t> F_Lane;
    ImmutableMapFactory<VRegKey, PM_Lane> F_VReg;
};

class PDist {
    PM_VReg Root;
    PMapFactories *F;

public:
    PDist(PMapFactories &Factories, PM_VReg R = PM_VReg()) : Root(R),
    F(&Factories) {}

    // Lookup helpers
    const int64_t *lookupLane(unsigned VReg, LaneBitmask M) const;

    // Update helpers
    PDist insertMin(unsigned VReg, LaneBitmask M, int64_t Stored) const;
    PDist clearLane(unsigned VReg, LaneBitmask M) const;
    PDist mergeRebased(const PDist &Succ, int64_t SuccEntryOff, int64_t
    EdgeWeight) const;
};

```

5. Usage in Analysis

- **Per-instruction snapshot:** `InstrVersion[I] = CurrRoot;` (just a pointer).
- **Operand use:** `Curr = Curr.insertMin(VReg, Mask, -Offset);`
- **Operand def:** `Curr = Curr.clearLane(VReg, Mask);`
- **Merge:** `Curr = Curr.mergeRebased(SuccRoot, EntryOff[Succ], Weight);`

Each change creates a new persistent root; old versions remain for queries.

6. Complexity

- **Update:** $O(\log \hat{K})$ per change ($\hat{K}$ = # distinct VRegs). Each operand touch does this.
- **Query:** $O(1)$ to fetch snapshot (root pointer), then $O(\log \hat{K})$ to lookup a specific vreg/lane.
- **Memory:** proportional to total changes $\cdot \log \hat{K}$, not $I \cdot \hat{K}$.

7. Pros / Cons

✓ Huge memory savings vs full snapshots. 3–10× less memory in typical cases. ✓ Queries remain simple — no replay, no recompute. ⚠ Slightly higher per-update cost ($O(\log \hat{K})$ vs $O(1)$ tick increment). ⚠ Requires integrating `ImmutableMap` into NUA and handling factories carefully.

8. Next Steps

- Prototype `PDist` wrapper around `ImmutableMap`.
- Replace `DenseMap` snapshots in NUA with `PDist` versions.
- Measure memory usage vs full snapshot baseline.
- Benchmark compile-time trade-offs on real AMDGPU workloads.

Summary: Persistent maps (`ImmutableMap`) allow us to keep *versioned snapshots of register distances* at every instruction without $\Theta(I \cdot \hat{K})$ blowup. This offers a principled way to combine correctness (instant snapshot queries) with manageable memory, at the cost of modest per-update overhead.